



# Laboratorio di Elettronica Digitale

## - VHDL Simulaiton -

Prof. Stefano Ricci



UNIVERSITÀ  
DEGLI STUDI  
FIRENZE

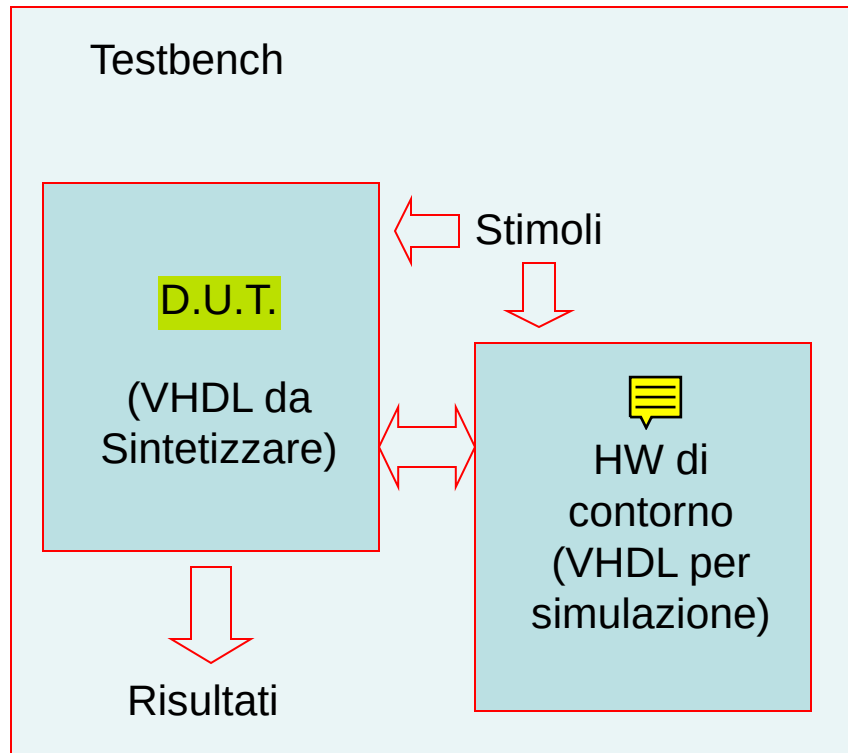
**DINFO**

DIPARTIMENTO DI  
INGEGNERIA  
DELL'INFORMAZIONE

Updated 02/2025

# TESTBENCH

- The VHDL code designed for synthesis is verified through simulations.
- Simulations are performed by building the “testbench” environment
- The Testbench is coded in VHDL as well.



➤ The testbench is a VHDL Entity with no I/O interface (no ports).

➤ The testbench entity includes the Device Under Test (DUT) and other possible entities that simulate other HW required for the test.

➤ The Testbench generates the “stimuli” signals and collects the DUT outputs that are analysed.

➤ Simulation tools, (like Modelsim from Mentor) allows the visualization of all the signals present in the DUT.



# TESTBENCH

## The Entity

```
library ieee;
use ieee.std_logic_1164.all;
use std.textio.all;
...
entity tstbench is
end tstbench;

architecture arc of tstbench is
    signal...

    ...
    component dut port(...);
    end component;
    ...
    component hw1 port(...);
    end component;
begin
    p1: process (...)
    end process p1;
    ...
    pn: process (...)
    end process pn;
    d1:dut port map (...);
    d2:hw1 port map (...);
end arc;
```

- The libraries needed for simulations are included (e.g. libraries for file access)
- testbench has no I/O ports
- The DUT and other sub-entities are declared and instantiated
- Stimuli signals are generated through processes
- The Testbench is NOT synthesized, thus a larger VHDL set can be used

# TESTBENCH

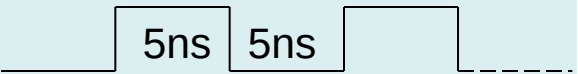
```
library ieee;  
use ieee.std_logic_1164.all;  
  
entity tstbench is  
end tstbench ;  
  
architecture arc of tstbench is  
    signal reset_tst, ck_tst, l_tst, u_tst: std_logic  
    component ck_div  
        port ( reset, ck, l : in std_logic;  
              u: out std_logic );  
    end component ;  
  
begin  
    ckp: process (...)  
    end process ckp;  
    resetp: process (...)  
    end process resetp;  
    lp: process (...)  
    end process lp;  
    d1: ck_div port map (  
        reset_tst, ck_tst, l_tst, u_tst);  
end arc;
```

Esempio: testbench di ck\_div

- Si dichiara componente sotto test
- Si definisce un segnale \*\_tst per ogni segnale in interfaccia
- Si usa un processo per ciascun stimolo (ingresso) da generare
- Si istanzia il componente

# TESTBENCH

```
ckp: process
begin
    ck_tst <= '0';
    wait for 5 ns;
    ck_tst <= '1';
    wait for 5 ns;
end process ckp;
```

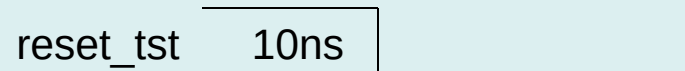
ck\_tst 

$T = 10 \text{ ns} \iff f = 100 \text{ MHz}$

- le istruzioni **wait** sono mutualmente esclusive rispetto alla **sensitivity list**
- ❖ la sensitivity list equivale ad un wait for “eventi sui segnali elencati”
- ❖ ad ogni sospensione le assegnazioni schedulate vengono eseguite
- ❖ **wait for time** sospende il processo per *time*

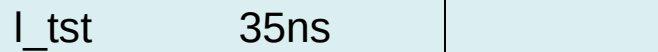
# TESTBENCH

```
resetp: process
begin
    reset_tst <= '1';
    wait for 10 ns;
    reset_tst <= '0';
    wait;
end process resetp;
```



Timing diagram for reset\_tst: The signal is high for 10ns and then transitions to low, remaining low indefinitely.

```
lp: process
begin
    l_tst <= '0';
    wait for 35 ns;
    l_tst <= '1';
    wait;
end process lp;
```



Timing diagram for l\_tst: The signal is low for 35ns and then transitions to high, remaining high indefinitely.

➤ **wait** (senza **for**...) sospende il processo indefinitamente